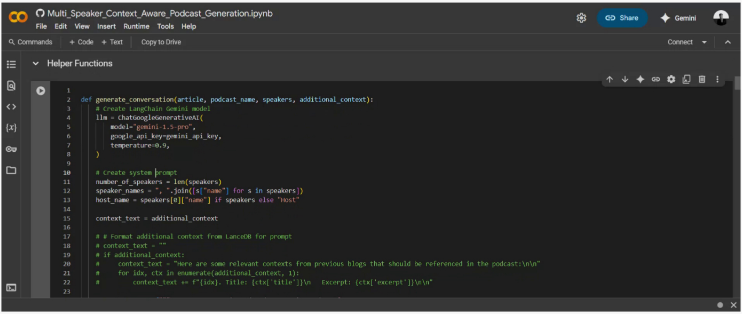
Creating an AI-Powered Podcast

> This document provides a comprehensive overview of transforming Medium blogs into an AI-driven podcast. The process involves extracting content from blogs, generating scripts, and producing audio with multiple speakers using advanced AI tools. The following steps guide you through each phase of this innovative podcast creation method.

Step 1

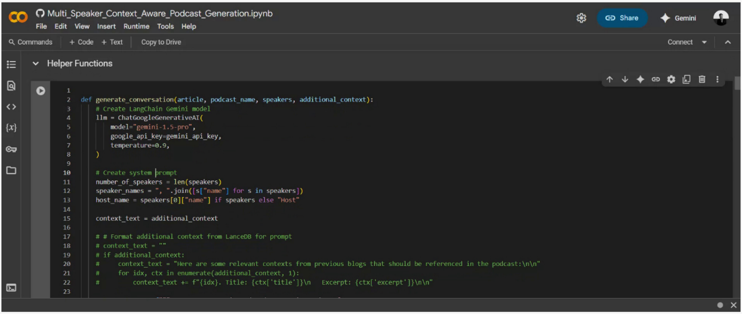
Initiate the process by creating a Colab notebook where you can write the necessary code to transform Medium blogs into a podcast using AI. The aim is to create an end-to-end AI podcast based on your Medium content.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File
Edit
View
Insert
Runtime
Tools
Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```python

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 2

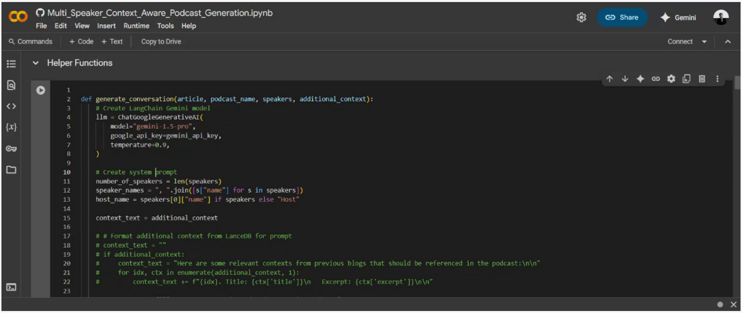
Start by retrieving blogs from your Medium profile. This will help create context around your previous blogs. The process is not merely about podcast generation; it involves generating a contextual AI podcast featuring multiple speakers, allowing flexibility in the number of speakers you choose.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 3

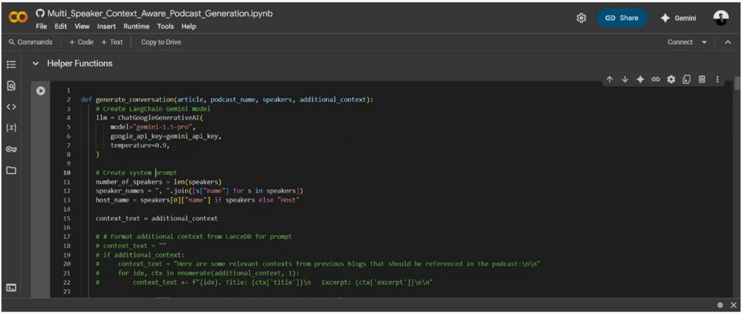
Podcasts typically feature two to three speakers. This tutorial covers the complete process. The technology stack includes LangChain, ElevenLabs for AI audio, Gemini for creating scripts, and LanceDB as a vector store for context retrieval from previous blogs, ensuring the model recognizes related topics.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 4

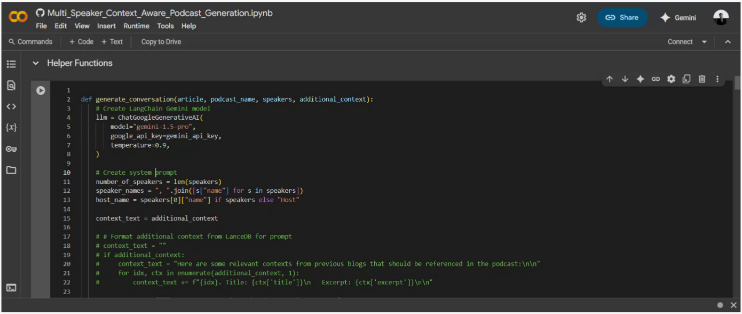
To maintain context from previous blogs, a vector database is created to store all relevant information. The initial step involves installing necessary libraries such as ElevenLabs and Smallest AI.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```
def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"
```

Step 5

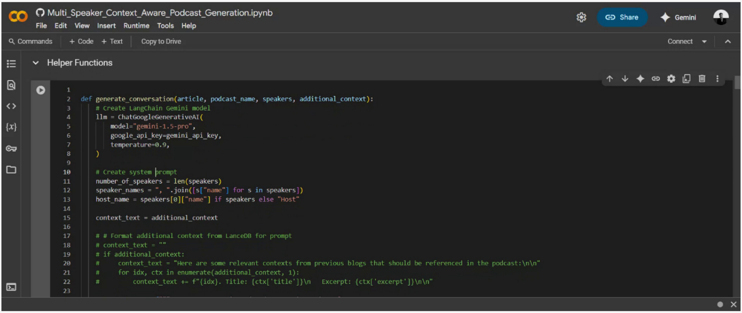
Use AI models from both ElevenLabs and Smallest AI to experiment with various voice models, including Indian voices. Quick installation commands are available, along with several helpful functions.



```
def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"
```

Step 6

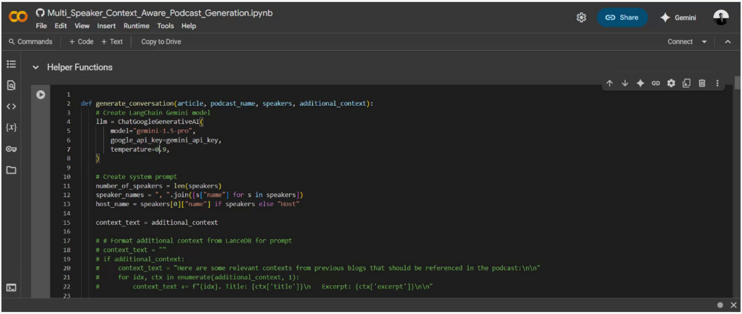
In the helper function section, you'll find multiple functions that assist in creating the podcast.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 7

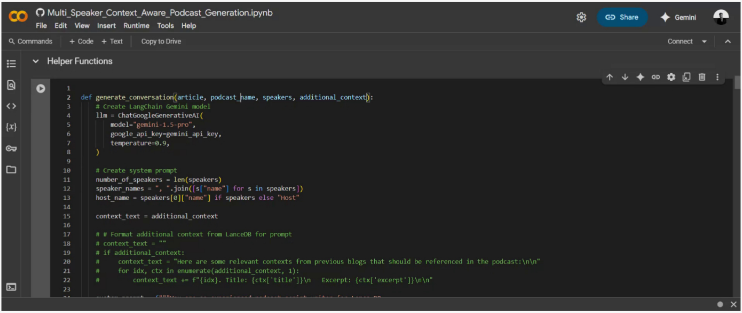
The first major function is `generate_conversation` , which is an LLM function. It takes the input article from which you want to generate the podcast.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 8

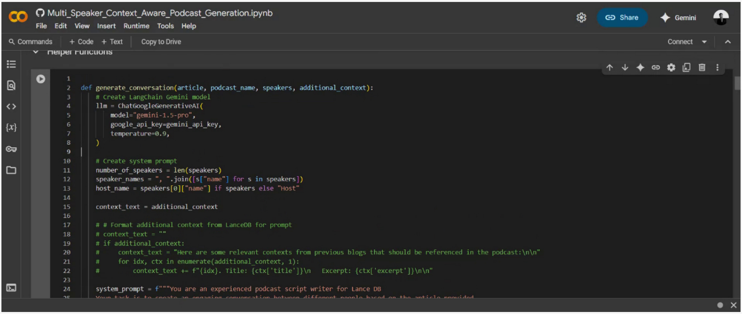
This function also requires the podcast name.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

```

Step 9

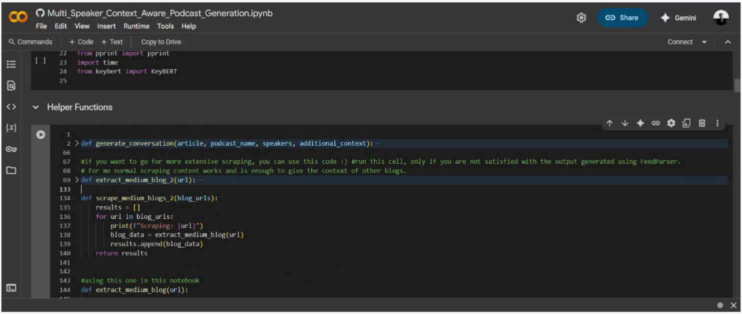
Additionally, specify the number of speakers and their names for your podcast.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```

def generate_conversation(article, podcast_name, speakers, additional_context):
    # Create LangChain Gemini model
    llm = ChatGoogleGenerativeAI(
        model="gemini-1.5-pro",
        google_api_key=gemini_api_key,
        temperature=0.9,
    )

    # Create system prompt
    number_of_speakers = len(speakers)
    speaker_names = ", ".join([s["name"] for s in speakers])
    host_name = speakers[0]["name"] if speakers else "Host"

    context_text = additional_context

    # # Format additional context from LanceDB for prompt
    # context_text = ""
    # if additional_context:
    #     context_text = "Here are some relevant contexts from previous blogs that should be referenced in the podcast:\n\n"
    #     for idx, ctx in enumerate(additional_context, 1):
    #         context_text += f"{idx}. Title: {ctx['title']}\n   Excerpt: {ctx['excerpt']}\n\n"

    system_prompt = f"""You are an experienced podcast script writer for Lance DB
```

Step 10

The podcast can feature greetings like "Hi Shresth" or "Hi Arjun." The `generate_conversation` function includes these details. Next, the process involves extracting Medium blogs from your profile.

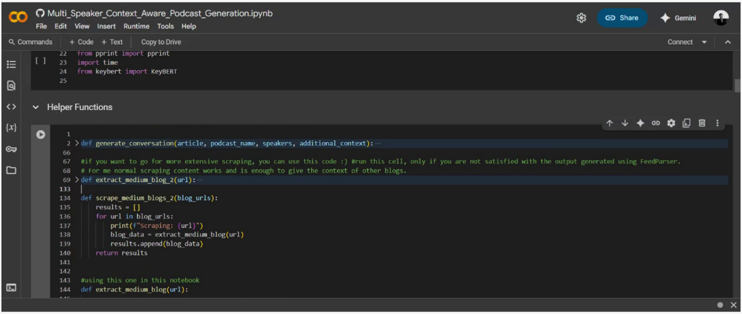

Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```
from pprint import pprint
import time
from keybert import KeyBERT

```

```

def generate_conversation(article, podcast_name, speakers, additional_context):···

#if you want to go for more extensive scraping, you can use this code :) #run this cell, only if you are not satisfied with the output generated using FeedParser.
# For me normal scraping content works and is enough to give the context of other blogs.
def extract_medium_blog_2(url):···

def scrape_medium_blogs_2(blog_urls):
    results = []
    for url in blog_urls:
        print(f"Scraping: {url}")
        blog_data = extract_medium_blog(url)
        results.append(blog_data)
    return results


#using this one in this notebook
def extract_medium_blog(url):
```

Step 11

For profiles not behind a paywall, the system scrapes the home page feed using the profile URL and creates a vector store.

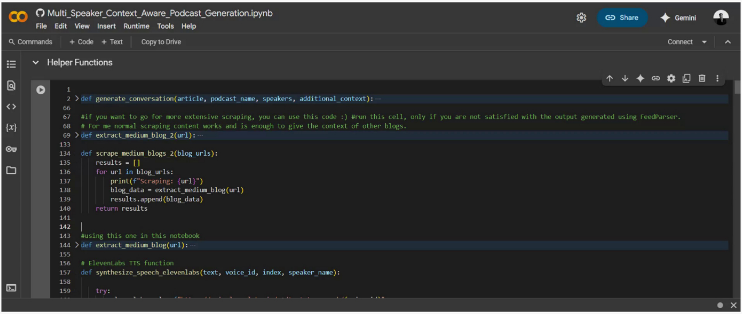

Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```
from pprint import pprint
import time
from keybert import KeyBERT

```

```

def generate_conversation(article, podcast_name, speakers, additional_context):···

#if you want to go for more extensive scraping, you can use this code :) #run this cell, only if you are not satisfied with the output generated using FeedParser.
# For me normal scraping content works and is enough to give the context of other blogs.
def extract_medium_blog_2(url):···

def scrape_medium_blogs_2(blog_urls):
    results = []
    for url in blog_urls:
        print(f"Scraping: {url}")
        blog_data = extract_medium_blog(url)
        results.append(blog_data)
    return results


#using this one in this notebook
def extract_medium_blog(url):
```

Step 12

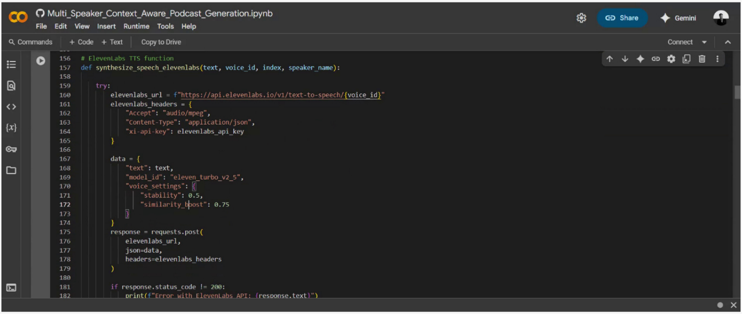
If your content is behind a paywall, limited information is tracked, capturing only the preview data.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
Helper Functions

```python

def generate_conversation(article, podcast_name, speakers, additional_context): ···

#if you want to go for more extensive scraping, you can use this code :) #run this cell, only if you are not satisfied with the output generated using FeedParser.
# For me normal scraping content works and is enough to give the context of other blogs.
def extract_medium_blog_2(url): ···

def scrape_medium_blogs_2(blog_urls):
    results = []
    for url in blog_urls:
        print(f"Scraping: {url}")
        blog_data = extract_medium_blog(url)
        results.append(blog_data)
    return results


#using this one in this notebook
def extract_medium_blog(url): ···

# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name):

    try:
```

Step 13

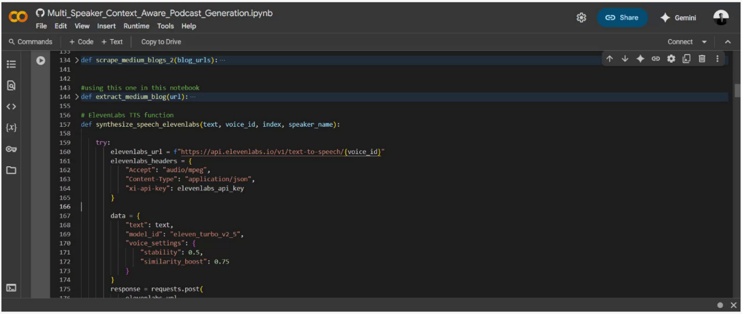
The next function is called `synthesize_speech_labs`.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```python
# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name):

    try:
        elevenlabs_url = f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}"
        elevenlabs_headers = {
            "Accept": "audio/mpeg",
            "Content-Type": "application/json",
            "xi-api-key": elevenlabs_api_key
        }

        data = {
            "text": text,
            "model_id": "eleven_turbo_v2_5",
            "voice_settings": {
                "stability": 0.5,
                "similarity_boost": 0.75
            }
        }
        response = requests.post(
            elevenlabs_url,
            json=data,
            headers=elevenlabs_headers
        )

        if response.status_code != 200:
            print(f"Error with ElevenLabs API: {response.text}")
```

Step 14

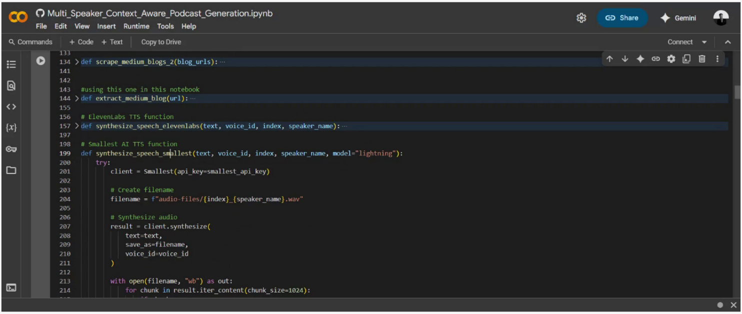
This function interacts with the ElevenLabs API, with the API key securely stored in the Colab notebook.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```python
def scrape_medium_blogs_2(blog_urls): ···


#using this one in this notebook
def extract_medium_blog(url): ···

# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name):

    try:
        elevenlabs_url = f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}"
        elevenlabs_headers = {
            "Accept": "audio/mpeg",
            "Content-Type": "application/json",
            "xi-api-key": elevenlabs_api_key
        }

        data = {
            "text": text,
            "model_id": "eleven_turbo_v2_5",
            "voice_settings": {
                "stability": 0.5,
                "similarity_boost": 0.75
            }
        }
        response = requests.post(
```

Step 15

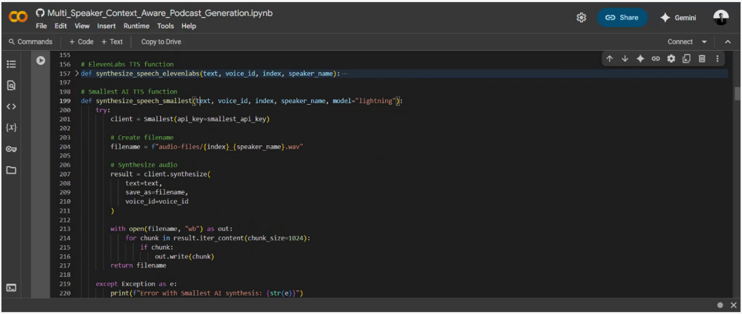
Similarly, `synthesize_speech_smallest` utilizes Smallest AI models.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```python
def scrape_medium_blogs_2(blog_urls): ···


#using this one in this notebook
def extract_medium_blog(url): ···

# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name): ···

# Smallest AI TTS function
def synthesize_speech_smallest(text, voice_id, index, speaker_name, model="lightning"):
    try:
        client = Smallest(api_key=smallest_api_key)

        # Create filename
        filename = f"audio-files/{index}_{speaker_name}.wav"

        # Synthesize audio
        result = client.synthesize(
            text=text,
            save_as=filename,
            voice_id=voice_id
        )

        with open(filename, "wb") as out:
            for chunk in result.iter_content(chunk_size=1024):
```

Step 16

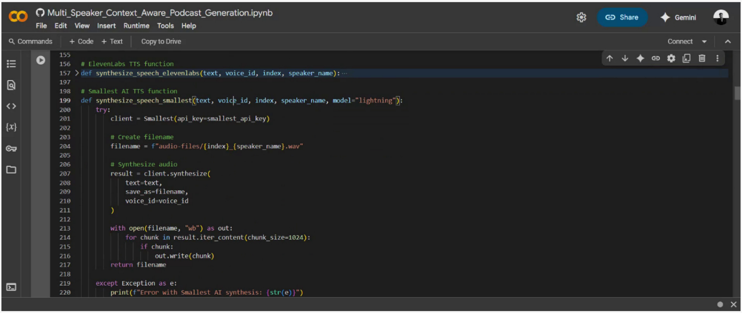
Smallest AI models are preferred for Indian voices. Pass the text for which you want to generate speech.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect

```

# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name):···

# Smallest AI TTS function
def synthesize_speech_smallest(text, voice_id, index, speaker_name, model="lightning"):
    try:
        client = Smallest(api_key=smallest_api_key)

        # Create filename
        filename = f"audio-files/{index}_{speaker_name}.wav"

        # Synthesize audio
        result = client.synthesize(
            text=text,
            save_as=filename,
            voice_id=voice_id
        )

        with open(filename, "wb") as out:
            for chunk in result.iter_content(chunk_size=1024):
                if chunk:
                    out.write(chunk)
        return filename

    except Exception as e:
        print(f"Error with Smallest AI synthesis: {str(e)}")
```

Step 17

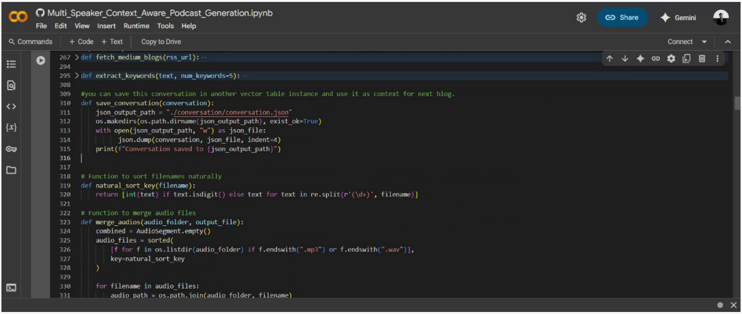
Include the voice ID, the index, and the speaker's name for accurate voice mapping.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect

```

# ElevenLabs TTS function
def synthesize_speech_elevenlabs(text, voice_id, index, speaker_name):···

# Smallest AI TTS function
def synthesize_speech_smallest(text, voice_id, index, speaker_name, model="lightning"):
    try:
        client = Smallest(api_key=smallest_api_key)

        # Create filename
        filename = f"audio-files/{index}_{speaker_name}.wav"

        # Synthesize audio
        result = client.synthesize(
            text=text,
            save_as=filename,
            voice_id=voice_id
        )

        with open(filename, "wb") as out:
            for chunk in result.iter_content(chunk_size=1024):
                if chunk:
                    out.write(chunk)
        return filename

    except Exception as e:
        print(f"Error with Smallest AI synthesis: {str(e)}")
```

Step 18

Finally, the `generate_audio` function combines both functions above. It processes each speaker's script to produce audio content sequentially.

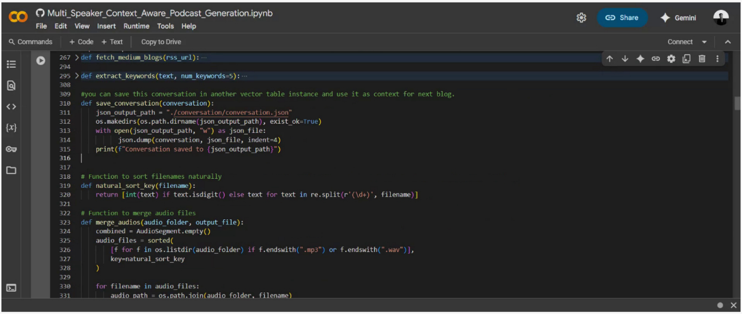


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```
def fetch_medium_blogs(rss_url): ...

def extract_keywords(text, num_keywords=5): ...

#you can save this conversation in another vector table instance and use it as context for next blog.
def save_conversation(conversation):
    json_output_path = "./conversation/conversation.json"
    os.makedirs(os.path.dirname(json_output_path), exist_ok=True)
    with open(json_output_path, "w") as json_file:
            json.dump(conversation, json_file, indent=4)
    print(f"Conversation saved to {json_output_path}")


# Function to sort filenames naturally
def natural_sort_key(filename):
    return [int(text) if text.isdigit() else text for text in re.split(r'(\d+)', filename)]

# Function to merge audio files
def merge_audios(audio_folder, output_file):
    combined = AudioSegment.empty()
    audio_files = sorted(
        [f for f in os.listdir(audio_folder) if f.endswith(".mp3") or f.endswith(".wav")],
        key=natural_sort_key
    )

    for filename in audio_files:
        audio_path = os.path.join(audio_folder, filename)
```

Step 19

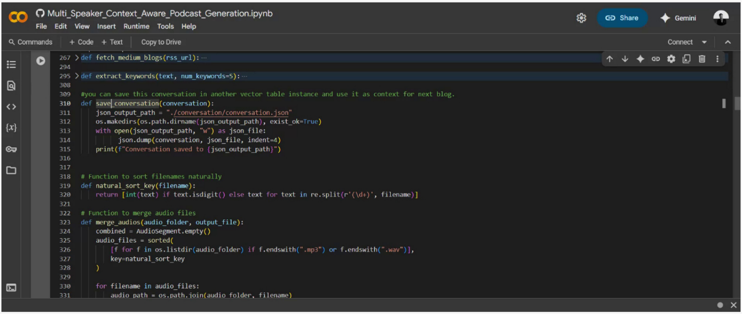
The `extract_keyword` function retrieves relevant information from previous blogs stored in the vector. The `save_conversation` function stores audio files in a JSON format.



```
def fetch_medium_blogs(rss_url): ...

def extract_keywords(text, num_keywords=5): ...

#you can save this conversation in another vector table instance and use it as context for next blog.
def save_conversation(conversation):
    json_output_path = "./conversation/conversation.json"
    os.makedirs(os.path.dirname(json_output_path), exist_ok=True)
    with open(json_output_path, "w") as json_file:
            json.dump(conversation, json_file, indent=4)
    print(f"Conversation saved to {json_output_path}")


# Function to sort filenames naturally
def natural_sort_key(filename):
    return [int(text) if text.isdigit() else text for text in re.split(r'(\d+)', filename)]

# Function to merge audio files
def merge_audios(audio_folder, output_file):
    combined = AudioSegment.empty()
    audio_files = sorted(
        [f for f in os.listdir(audio_folder) if f.endswith(".mp3") or f.endswith(".wav")],
        key=natural_sort_key
    )

    for filename in audio_files:
        audio_path = os.path.join(audio_folder, filename)
```

Step 20

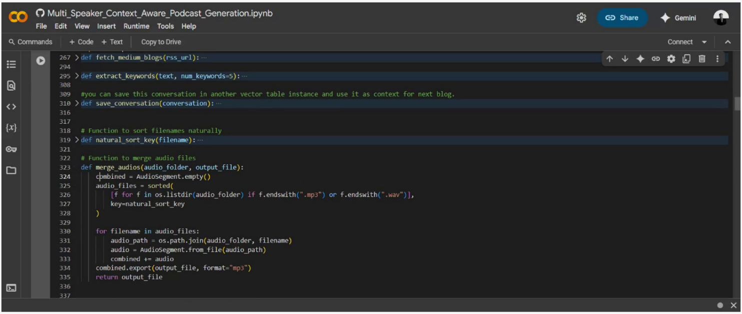
Save the conversation script in a JSON file format.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```
def fetch_medium_blogs(rss_url): ···

def extract_keywords(text, num_keywords=5): ···

#you can save this conversation in another vector table instance and use it as context for next blog.
def save_conversation(conversation):
    json_output_path = "./conversation/conversation.json"
    os.makedirs(os.path.dirname(json_output_path), exist_ok=True)
    with open(json_output_path, "w") as json_file:
        json.dump(conversation, json_file, indent=4)
    print(f"Conversation saved to {json_output_path}")


# Function to sort filenames naturally
def natural_sort_key(filename):
    return [int(text) if text.isdigit() else text for text in re.split(r'(\d+)', filename)]

# Function to merge audio files
def merge_audios(audio_folder, output_file):
    combined = AudioSegment.empty()
    audio_files = sorted(
        [f for f in os.listdir(audio_folder) if f.endswith(".mp3") or f.endswith(".wav")],
        key=natural_sort_key
    )

    for filename in audio_files:
        audio_path = os.path.join(audio_folder, filename)
```

Step 21

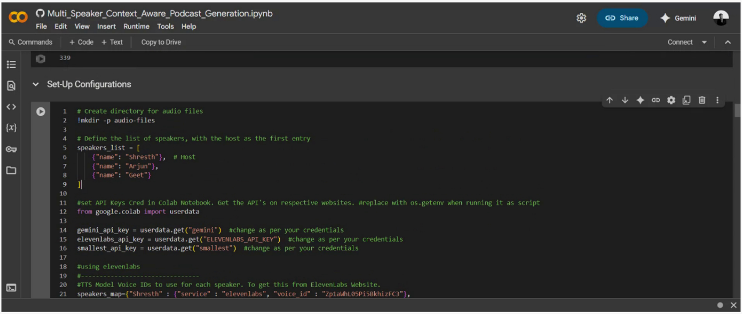
The `merge_audio` function compiles multiple audio files into a complete podcast.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```
def fetch_medium_blogs(rss_url): ···

def extract_keywords(text, num_keywords=5): ···

#you can save this conversation in another vector table instance and use it as context for next blog.
def save_conversation(conversation): ···


# Function to sort filenames naturally
def natural_sort_key(filename): ···

# Function to merge audio files
def merge_audios(audio_folder, output_file):
    combined = AudioSegment.empty()
    audio_files = sorted(
        [f for f in os.listdir(audio_folder) if f.endswith(".mp3") or f.endswith(".wav")],
        key=natural_sort_key
    )

    for filename in audio_files:
        audio_path = os.path.join(audio_folder, filename)
        audio = AudioSegment.from_file(audio_path)
        combined += audio
    combined.export(output_file, format="mp3")
    return output_file
```

Step 22

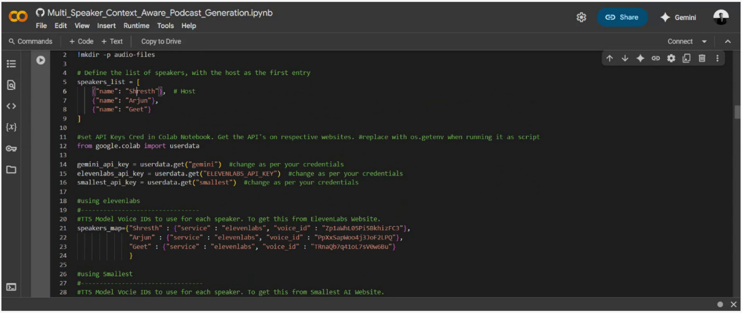
Determine the number of speakers for your podcast in a configuration setup.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect
339
Set-Up Configurations

```
# Create directory for audio files
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
```

Step 23

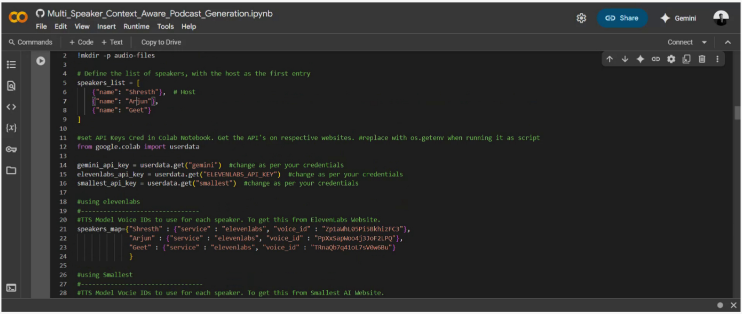
Choose different voices for each speaker, such as Shresth, Arjun, or Keith.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
#-------------------------------
#TTS Model Vocie IDs to use for each speaker. To get this from Smallest AI Website.
```

Step 24

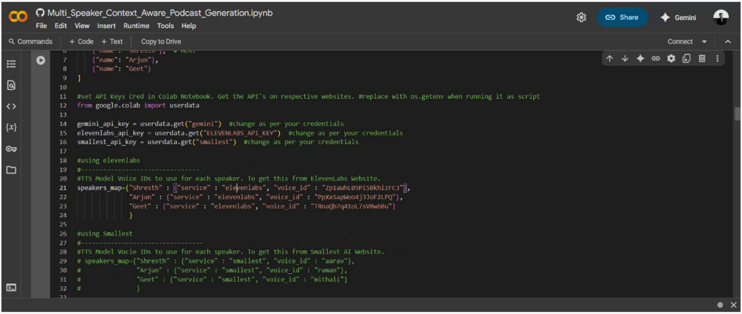
Assign distinct voices to each speaker, such as Shresth, Arjun, or Keith.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect

```
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#--------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
#---------------------------------
#TTS Model Vocie IDs to use for each speaker. To get this from Smallest AI Website.
```

Step 25

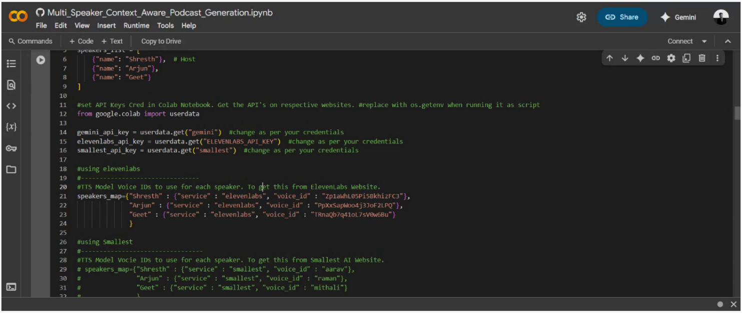
Use the ElevenLabs API for voice assignments and retrieve the appropriate voice ID.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect

```
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#--------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
#---------------------------------
#TTS Model Vocie IDs to use for each speaker. To get this from Smallest AI Website.
# speakers_map={"Shresth" : {"service" : "smallest", "voice_id" : "aarav"},
#               "Arjun" : {"service" : "smallest", "voice_id" : "raman"},
#               "Geet" : {"service" : "smallest", "voice_id" : "mithali"}
#               }
```

Step 26

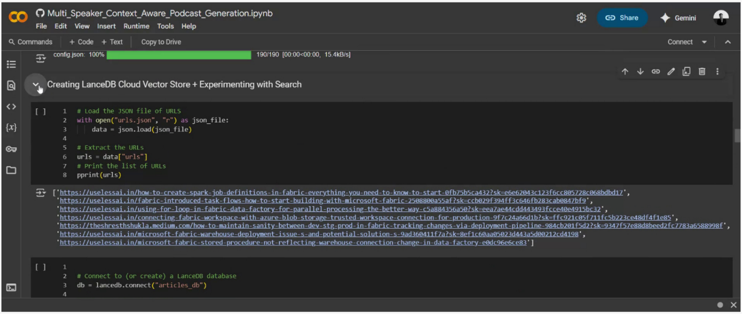
If using Smallest AI, select the corresponding voice ID based on the speaker's preference.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands
+ Code
+ Text
Copy to Drive
Share
Gemini
Connect

```python
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#-------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
#-------------------------------
#TTS Model Vocie IDs to use for each speaker. To get this from Smallest AI Website.
# speakers_map={"Shresth" : {"service" : "smallest", "voice_id" : "aarav"},
#               "Arjun" : {"service" : "smallest", "voice_id" : "raman"},
#               "Geet" : {"service" : "smallest", "voice_id" : "mithali"}
```

Step 27

Ensure the API key is configured correctly. The first step involves creating a vector store from your profile.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands
+ Code
+ Text
Copy to Drive
Share
Gemini
Connect

```
config.json: 100%  190/190 [00:00<00:00, 15.4kB/s]
```

Creating LanceDB Cloud Vector Store + Experimenting with Search

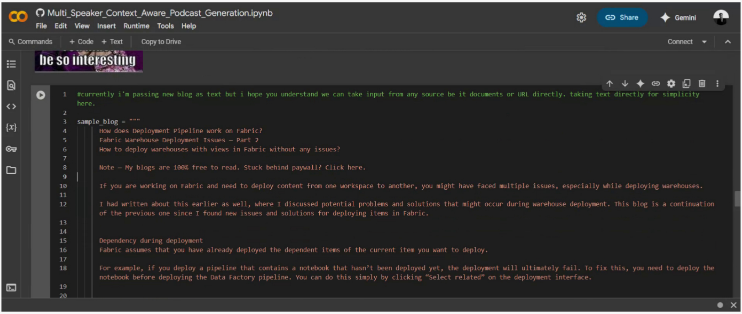
```python
# Load the JSON file of URLS
with open("urls.json", "r") as json_file:
    data = json.load(json_file)

# Extract the URLs
urls = data["urls"]
# Print the list of URLs
pprint(urls)
```

```
['https://uselessai.in/how-to-create-spark-job-definitions-in-fabric-everything-you-need-to-know-to-start-0fb75b5ca432?sk=e6e62043c123f6cc805728c068bdbd17',
 'https://uselessai.in/fabric-introduced-task-flows-how-to-start-building-with-microsoft-fabric-2508800a55af?sk=ccb029f394ff3c646fb283cab0847bf9',
 'https://uselessai.in/using-for-loop-in-fabric-data-factory-for-parallel-processing-the-better-way-c5a884356a50?sk=eea7ae44cdd443493fcce40e4915bc32',
 'https://uselessai.in/connecting-fabric-workspace-with-azure-blob-storage-trusted-workspace-connection-for-production-9f7c24a66d1b?sk=ffc921c05f711fc5b223ce48df4f1e85',
 'https://theshresthshukla.medium.com/how-to-maintain-sanity-between-dev-stg-prod-in-fabric-tracking-changes-via-deployment-pipeline-984cb201f5d2?sk=9347f57e88d8beed2fc7783a6588998f',
 'https://uselessai.in/microsoft-fabric-warehouse-deployment-issue-s-and-potential-solution-s-9ad360411f7a?sk=8ef1c60aa05023d443a5d00212cd4198',
 'https://uselessai.in/microsoft-fabric-stored-procedure-not-reflecting-warehouse-connection-change-in-data-factory-e0dc96e6ce83']
```

```python

# Connect to (or create) a LanceDB database
db = lancedb.connect("articles_db")

```

Step 28

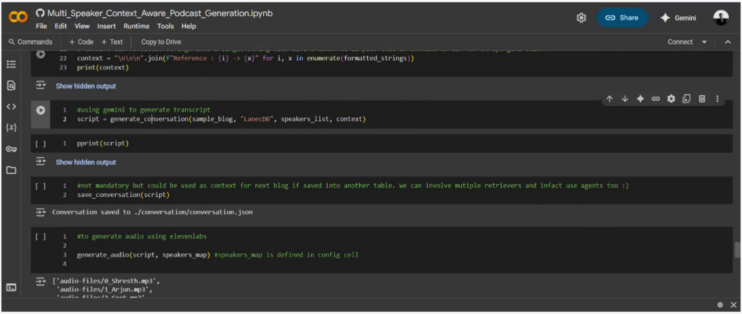
Once the vector store and API key are set, input your text blog or URL. Extract relevant keywords to search for similar topics within your context.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect
be so interesting

```
#currently i'm passing new blog as text but i hope you understand we can take input from any source be it documents or URL directly. taking text directly for simplicity here.

sample_blog = """
    How does Deployment Pipeline work on Fabric?
    Fabric Warehouse Deployment Issues — Part 2
    How to deploy warehouses with views in Fabric without any issues?

    Note — My blogs are 100% free to read. Stuck behind paywall? Click here.

    If you are working on Fabric and need to deploy content from one workspace to another, you might have faced multiple issues, especially while deploying warehouses.

    I had written about this earlier as well, where I discussed potential problems and solutions that might occur during warehouse deployment. This blog is a continuation of the previous one since I found new issues and solutions for deploying items in Fabric.


    Dependency during deployment
    Fabric assumes that you have already deployed the dependent items of the current item you want to deploy.

    For example, if you deploy a pipeline that contains a notebook that hasn't been deployed yet, the deployment will ultimately fail. To fix this, you need to deploy the notebook before deploying the Data Factory pipeline. You can do this simply by clicking "Select related" on the deployment interface.
```

Step 29

Utilize the created helper functions to generate conversations and audio.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect
Show hidden output
Show hidden output

```
context = "\n\n\n".join(f"Reference : {i} -> {x}" for i, x in enumerate(formatted_strings))
print(context)
```

```
#using gemini to generate transcript
script = generate_conversation(sample_blog, "LanecDB", speakers_list, context)
```

```
pprint(script)
```

```
#not mandatory but could be used as context for next blog if saved into another table. We can involve mutiple retrievers and infact use agents too :)
save_conversation(script)
```

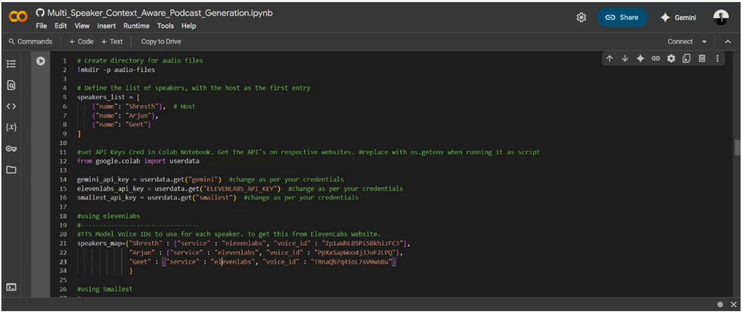
```
Conversation saved to ./conversation/conversation.json
```

```
#to generate audio using elevenlabs

generate_audio(script, speakers_map) #speakers_map is defined in config cell

```

```
['audio-files/0_Shresth.mp3',
 'audio-files/1_Arjun.mp3',
```

Step 30

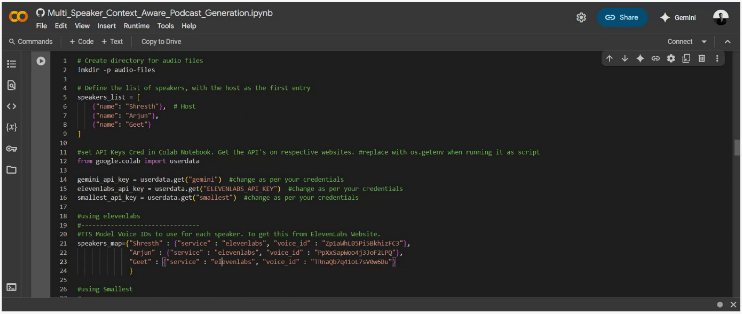
Employ these functions to generate conversations, create multiple audios, and produce the final podcast. The AI podcast can feature multiple speakers, offering flexibility in selection.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File
Edit
View
Insert
Runtime
Tools
Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```python
# Create directory for audio files
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#-------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
```

Step 31

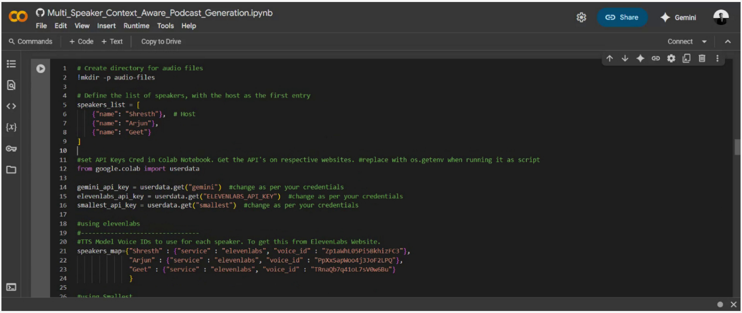
Include multiple voice options, accommodating various genders.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File
Edit
View
Insert
Runtime
Tools
Help
Share
Gemini
Commands
+ Code
+ Text
Copy to Drive
Connect

```python
# Create directory for audio files
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#-------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
```

Step 32

Incorporate both male and female voices, integrating any LLM or AI voice you prefer in a customizable manner.

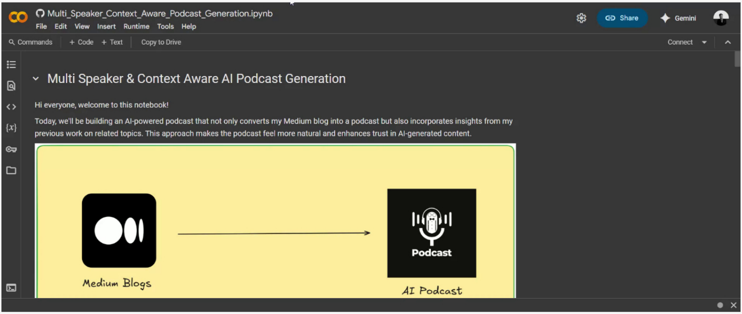

Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect

```
# Create directory for audio files
!mkdir -p audio-files

# Define the list of speakers, with the host as the first entry
speakers_list = [
    {"name": "Shresth"},  # Host
    {"name": "Arjun"},
    {"name": "Geet"}
]

#set API Keys Cred in Colab Notebook. Get the API's on respective websites. #replace with os.getenv when running it as script
from google.colab import userdata

gemini_api_key = userdata.get("gemini")  #change as per your credentials
elevenlabs_api_key = userdata.get("ELEVENLABS_API_KEY")  #change as per your credentials
smallest_api_key = userdata.get("smallest")  #change as per your credentials

#using elevenlabs
#--------------------------------
#TTS Model Voice IDs to use for each speaker. To get this from ElevenLabs Website.
speakers_map={"Shresth" : {"service" : "elevenlabs", "voice_id" : "Zp1aWhL05Pi5BkhizFC3"},
              "Arjun" : {"service" : "elevenlabs", "voice_id" : "PpXxSapWoo4j3JoF2LPQ"},
              "Geet" : {"service" : "elevenlabs", "voice_id" : "TRnaQb7q41oL7sV0w6Bu"}
              }

#using Smallest
```

Step 33

Explore the Colab notebook for a quick overview of using these functions to create your podcast.

Thank you for your attention.


Multi_Speaker_Context_Aware_Podcast_Generation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Copy to Drive
Share
Gemini
Connect
Podcast
Medium Blogs
AI Podcast

Multi Speaker & Context Aware AI Podcast Generation

Hi everyone, welcome to this notebook!

Today, we'll be building an AI-powered podcast that not only converts my Medium blog into a podcast but also incorporates insights from my previous work on related topics. This approach makes the podcast feel more natural and enhances trust in AI-generated content.